WEEK – 12

1# Write a PL/SQL stored procedure to display “Hello World”.

```
CREATE OR REPLACE PROCEDURE hello_world AS
BEGIN
    DBMS_OUTPUT.PUT_LINE('Hello World');
END;

BEGIN
    hello_world;
END;
```

Results | Explain | Describe | Saved SQL | History

```
Hello World

Statement processed.

0.00 seconds
```

2# Write a PL/SQL stored procedure to calculate the sum of digits at even place of a number entered through users.

```
CREATE OR REPLACE PROCEDURE sum_even_place_digits(num IN NUMBER) AS
    n NUMBER := num;
    rem NUMBER;
    pos NUMBER := 1;
    sum_even NUMBER := 0;
BEGIN
    WHILE n > 0 LOOP
        rem := MOD(n, 10);

        IF MOD(pos, 2) = 0 THEN
            sum_even := sum_even + rem;
        END IF;

        n := FLOOR(n / 10);
        pos := pos + 1;
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Sum of digits at even places = ' || sum_even);
END;
```

Results | Explain | Describe | Saved SQL | History

```
Procedure created.

0.03 seconds
```

```
BEGIN
    sum_even_place_digits(123456);
END;
```

3# Write a PL/SQL stored procedure to count prime and composite numbers entered by the user.

```oracle
CREATE OR REPLACE PROCEDURE count_prime_composite(n1 NUMBER, n2 NUMBER, n3 NUMBER) AS
    prime_count NUMBER := 0;
    composite_count NUMBER := 0;

    FUNCTION is_prime(n NUMBER) RETURN BOOLEAN IS
        i NUMBER;
    BEGIN
        IF n < 2 THEN
            RETURN FALSE;
        END IF;

        FOR i IN 2..n-1 LOOP
            IF MOD(n, i) = 0 THEN
                RETURN FALSE;
            END IF;
        END LOOP;

        RETURN TRUE;
    END;

BEGIN
    IF is_prime(n1) THEN prime_count := prime_count + 1; ELSE composite_count := composite_count + 1; END IF;
    IF is_prime(n2) THEN prime_count := prime_count + 1; ELSE composite_count := composite_count + 1; END IF;
    IF is_prime(n3) THEN prime_count := prime_count + 1; ELSE composite_count := composite_count + 1; END IF;

    DBMS_OUTPUT.PUT_LINE('Prime Count = ' || prime_count);
    DBMS_OUTPUT.PUT_LINE('Composite Count = ' || composite_count);
END;
```

```oracle
BEGIN
    count_prime_composite(7, 10, 15);
END;
```

```
Prime Count = 1
Composite Count = 2

Statement processed.

0.01 seconds
```

4# Write a PL-SQL function to compare three numbers entered through users and display them in ascending and descending orders.

```
CREATE OR REPLACE FUNCTION compare_numbers(a NUMBER, b NUMBER, c NUMBER)
RETURN VARCHAR2 IS
BEGIN
    DBMS_OUTPUT.PUT_LINE('Ascending: ' ||
        LEAST(a,b,c) || ' ' ||
        (a+b+c - LEAST(a,b,c) - GREATEST(a,b,c)) || ' ' ||
        GREATEST(a,b,c));

    DBMS_OUTPUT.PUT_LINE('Descending: ' ||
        GREATEST(a,b,c) || ' ' ||
        (a+b+c - LEAST(a,b,c) - GREATEST(a,b,c)) || ' ' ||
        LEAST(a,b,c));

    RETURN 'Done';
END;
```

```
BEGIN
    DBMS_OUTPUT.PUT_LINE(compare_numbers(30, 10, 20));
END;
```

Results | **Explain** | **Describe** | **Saved**

```
Ascending: 10 20 30
Descending: 30 20 10
Done

Statement processed.
```

0.00 seconds

5# Write a PL/SQL function that performs simple arithmetic like Addition, Subtraction, Multiplication & Division of input numbers

```oracle
CREATE OR REPLACE FUNCTION arithmetic_ops(a NUMBER, b NUMBER, op VARCHAR2)
RETURN NUMBER IS
BEGIN
    IF op = 'ADD' THEN
        RETURN a + b;

    ELSIF op = 'SUB' THEN
        RETURN a - b;

    ELSIF op = 'MUL' THEN
        RETURN a * b;

    ELSIF op = 'DIV' THEN
        IF b = 0 THEN
            RETURN NULL;
        ELSE
            RETURN a / b;
        END IF;

    ELSE
        RETURN NULL;
    END IF;
END;
```

```oracle
BEGIN
    DBMS_OUTPUT.PUT_LINE(arithmetic_ops(10, 5, 'ADD'));
    DBMS_OUTPUT.PUT_LINE(arithmetic_ops(10, 5, 'SUB'));
    DBMS_OUTPUT.PUT_LINE(arithmetic_ops(10, 5, 'MUL'));
    DBMS_OUTPUT.PUT_LINE(arithmetic_ops(10, 5, 'DIV'));
END;
```

Results | Explain | Describe | Saved SQL | History

```
15
5
50
2

Statement processed.
```